Programming HP Robots Otto with MicroPython

Developer Guide for Thonny IDE with ESP32-WROOM

HP Robots Otto was originally designed to run MicroPython, leveraging the ESP32 REPL for fast iteration and a beginner-friendly workflow.

Using Thonny IDE, you can write, run, and test code instantly on the robot without reflashing, making development much smoother and educational.

MicroPython is recommended for new users, classrooms, and rapid prototyping. While Arduino C++ provides deeper control, it requires full board erasing and flashing each time you switch languages.

You can move between MicroPython and Arduino, but switching back to MicroPython requires reflashing MicroPython firmware onto the ESP32-WROOM.

1. Requirements

- Thonny IDE (with MicroPython support)
- MicroPython firmware for ESP32-WROOM
- USB cable for connection
- Install 'esptool' if flashing manually

2. Setting Up MicroPython on Otto

1. Download ESP32-MicroPython firmware 1.6.1 (.bin)
2. Connect Otto to your computer via USB
3. Thonny → Tools → Options → Interpreter → MicroPython (ESP32)
4. Click 'Install MicroPython' to flash automatically

3. Basic MicroPython Examples

3.1 Built-in LED (GPIO 2)

```
from machine import Pin
import time
led = Pin(2, Pin.OUT)
while True:
    led.value(1)
```

```
    time.sleep(0.5)
    led.value(0)
    time.sleep(0.5)
```

3.2 NeoPixel Ring (GPIO 4)

```
from machine import Pin
import neopixel, time
np = neopixel.NeoPixel(Pin(4), 13)
while True:
    for i in range(13):
        np[i] = (255,0,0)
        np.write()
        time.sleep(0.05)
    for i in range(13):
        np[i] = (0,0,0)
    np.write()
```

3.3 OLED Display (I2C GPIO 21/22)

```
from machine import Pin, I2C
import ssd1306
i2c = I2C(0, scl=Pin(22), sda=Pin(21))
d = ssd1306.SSD1306_I2C(128, 64, i2c)
d.text('Hello Otto!', 0, 0)
d.show()
```

3.4 Digital Encoder (GPIO 16)

```
from machine import Pin
import time
enc = Pin(16, Pin.IN, Pin.PULL_UP)
count = 0

def handler(pin):
    global count
    count += 1

enc.irq(trigger=Pin.IRQ_FALLING, handler=handler)

while True:
    print(count)
    time.sleep(0.5)
```

4. Running Code in Thonny

1. Open Thonny, ensure MicroPython (ESP32) interpreter is selected
2. Write code in the editor and click Run → executes instantly over REPL
3. Save to device as main.py to auto-run at boot